Bookshelf Software:

Project Deliverable Final Report

Team 10 - The Librarians

Github Repository

Software Engineering

CS3354.002

Delegation of Tasks:

1. Jayden Boomer:
    - Created the github repository
    - Added all team members to the repository
    - Pgs. 8-16 : Collaborated on sequence diagrams
    - Committed the report to the repository
    - Pg.19 :Project Scheduling

2. Manuel De La Cruz
    - Pg.7 : Create Use Case Diagram
    - Pg. 28 : Conclusion

3. Andy Do
    - Ensure team project repository URL is included
    - Created test code for sample unit
    - Pgs. 22 - 24: Created test plan and presented results of sample unit code

4. Samuel Green
    - Added README file to Github
    - Pgs. 8-16 : Collaborated on sequence diagrams
    - Pg. 18: Created Architectural Design after team decided on best pattern to use

5. Sharon Kopuri
    - Pgs. 4-6 : List of software requirements including : Functional & Non functional requirements.
    - Pgs.19 - 21 : Cost, Effort and Pricing Estimation

6. Eli Mendez
    - Project scope pdf commit on github

7. Cassidy Peña
    - Pg.3 : Addressing Feedback
    - Pg.4 : Software Process Model employed
    - Pg.17 : Class Diagram
    - Pg. 25-27 : Comparison to other work

You can make assumptions, even make up government/country-based rules, requirements to be able to provide one for each

We did not make any assumptions nor make up government / country-based rules or requirements in the project.

Functional & Non-Functional Requirements - Assumptions made:

- Single-user, offline desktop app (Windows/macOS), local DB; no cloud
- Scope = **PDF + TXT** only; no OCR/DRM bypass; annotations/bookmarks stored in our Database, not the files.
- One watched the "Books" folder; many-to-many categories that don't delete books.
- Performance targets assume ~**1,000 books/3 GB** on mid-range SSD machines; times are warm-state.
- Core features: import, organize, search, read/nav, bookmarks, annotations, display prefs
- Security: encryption at rest via OS keystore
- Reliability: crash-safe imports; daily local backups.
- Out of scope: cloud sync, multi-device

Addressing the feedback provided from Course Team Project Proposal:

-List what you are doing / planning to do regarding the feedback provided for your project proposal

The feedback received for our proposal is, "It is unclear what exactly chapter extraction and traversal is from the proposed implementation.". Regarding this feedback we will make sure to make things more clear on their function in our software.

To make it clear, chapter extraction implies extracting the chapters in a pdf or text file and making a table of content for that file. On the other hand, chapter traversal is the ability to traverse through the chapters of a file using the table of contents generated by the chapter extraction. These two functionalities make the pdf or text file a more smooth and easy user experience.

Describe which software process model is employed in your project and why it was the choice. (Ch 2)

The software process model employed in our project is the Incremental Software model, one of the most common approaches. Meaning we are developing the software incrementally which is the best for our bookshelf software because our software is for students made by students. Our priority is for our software to be the best it can be for our customers / audience. By using the incremental process model we get to have feedback from the users and are able to change it for their liking, incrementally[9]. Finally, this model provides us with a more rapid delivery and deployment of useful software to the customers which is something we are prioritizing.

In the end, the software process model we are employing is the Incremental model because of its easy feedback it provides and its rapid delivery to the users.

List of software requirements including:

a. Functional requirements

1. **Import & Library Sync:**
    - The system shall scan the configured "Books" folder for `.pdf` and `.txt` files.
    - The system shall import each discovered file into the library.
    - The system shall complete scanning and importing of up to 500 files within 10 seconds on the baseline machine.
    - The system shall ignore duplicate files that correspond to an already imported book.
2. **Organize by Categories:**
    - The system shall allow the user to create a category.
    - The system shall allow the user to rename a category.
    - The system shall allow the user to delete a category.
    - The system shall allow the user to assign and unassign a book to/from a category.
    - The system shall not delete any books when a category is deleted.
3. **Text & Metadata Search:**
    - The system shall allow the user to enter a text query.
    - The system shall search titles, authors, and full text of .txt and text-selectable .pdf books.
    - The system shall return search results for a library of up to 1,000 books within 1 second on the baseline machine.
    - The system shall display, for each result, the book title, author, and a short text snippet.
4. **Reader Navigation:**

- The system shall allow the user to go to the next page and previous page.
  The system shall allow the user to jump directly to a specified page number.
- The system shall allow the user to follow a table-of-contents link to its target page.
- The system shall complete any page navigation action within 200 ms after the file is open on the baseline machine.

5. **Bookmarks:**
   - The system shall allow the user to add a bookmark at the current reading position.
   - The system shall allow the user to view a list of bookmarks for the current book.
   - The system shall allow the user to delete a bookmark.
   - The system shall allow the user to jump to a bookmark from the bookmark list.
   - The system shall persist bookmarks across app restarts.
6. **Annotations:**
   - The system shall allow the user to add an annotation to a page.
   - The system shall allow the user to view annotations on a page.
   - The system shall allow the user to edit an existing annotation.
   - The system shall allow the user to delete an annotation.
   - The system should allow the user to export all annotations for a book to a JSON file.
7. **Display Preferences:**
   - The system shall allow the user to switch between day mode and night mode.
   - The system shall allow the user to adjust font size for reflowable text.
   - The system shall allow the user to zoom in and out for PDF books.
   - The system shall persist display preferences per book across app restarts.

b. Non-functional requirements

1. **Product requirements**
   a. **Performance/Efficiency**
      - The system shall open a 50 MB PDF to the first visible page within 2.0 seconds on the baseline hardware.
      - The system shall complete a full-library re-index for 1,000 books or 3 GB within 60 seconds on the baseline hardware.

   b. **Dependability/Reliability**
      - The system **shall** achieve a **crash-free session rate ≥ 99.5%** over 1,000 test sessions.
      - The system **shall** maintain a consistent library state if an import operation is interrupted.

   c. **Security Requirements**

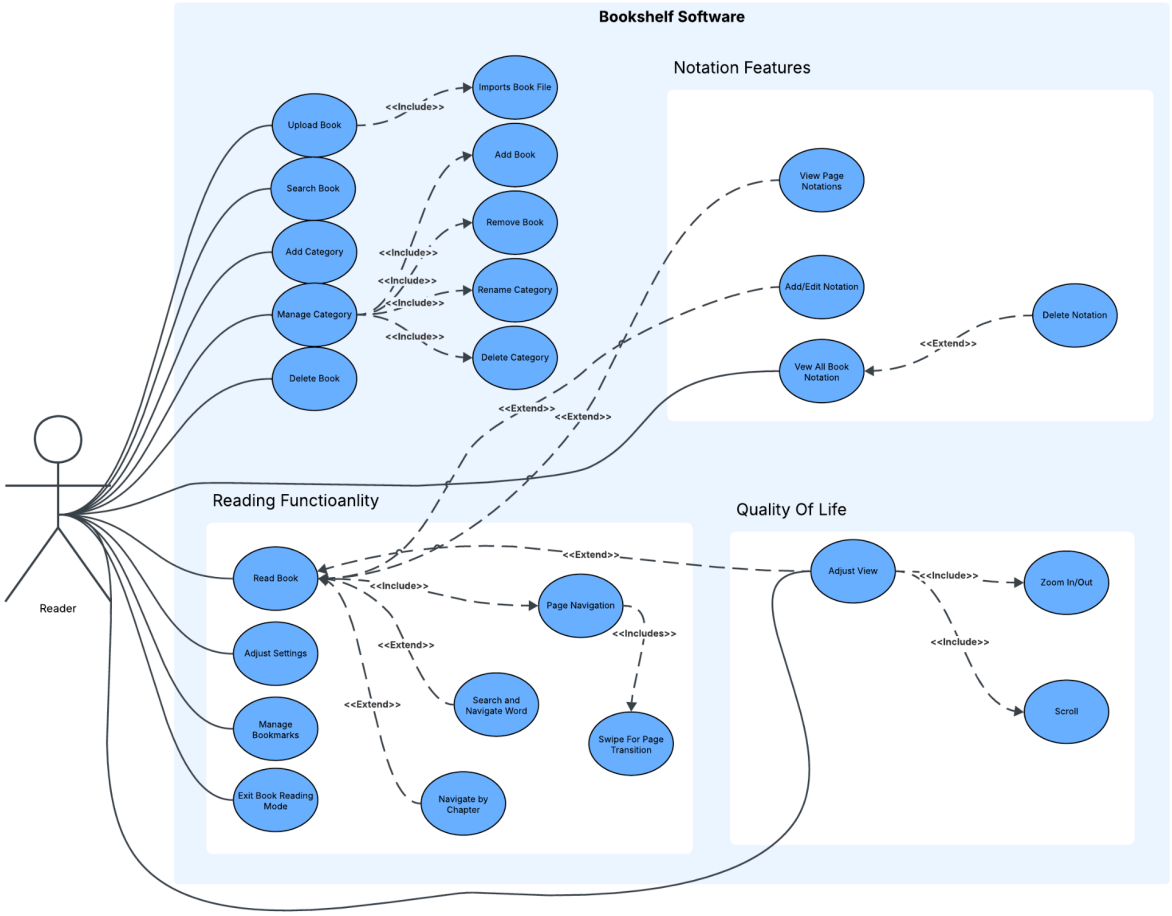
- The system **shall** encrypt all local data (library database, annotations, settings) at rest using a strong industry-standard encryption algorithm (e.g., AES-256).
- The system **shall not** perform any **network access** unless the user has explicitly opted in.

2. **Organizational requirements**
    a. **Usability**
        - The system shall allow at least 90% of novice users ($n \geq 10$) to complete an initial task sequence (import a book, create a category, add a bookmark) without help within 3 minutes.
        - The system shall provide keyboard navigation for all interactive controls.
        - The system shall provide screen-reader-friendly labels for all actionable controls.
    b. **Environmental (Operating Constraints)**
        - The system shall support Windows 10/11 and macOS 13+.
        - The system shall provide full core functionality offline.
    c. **Operational Requirements**
        - The system shall perform an automatic daily local backup of the library database and annotations.
        - The system shall restore the library database and annotations from the most recent backup within 30 seconds.

3. **External requirements**
    a. **Regulatory Requirements**
        - The system shall honor OS-level data-deletion requests and provide a “Delete all data” function that wipes all local stores within 5 seconds on baseline hardware.
        - NFR-7.2 The system shall not attempt to bypass DRM or other copy-protection mechanisms on PDFs.

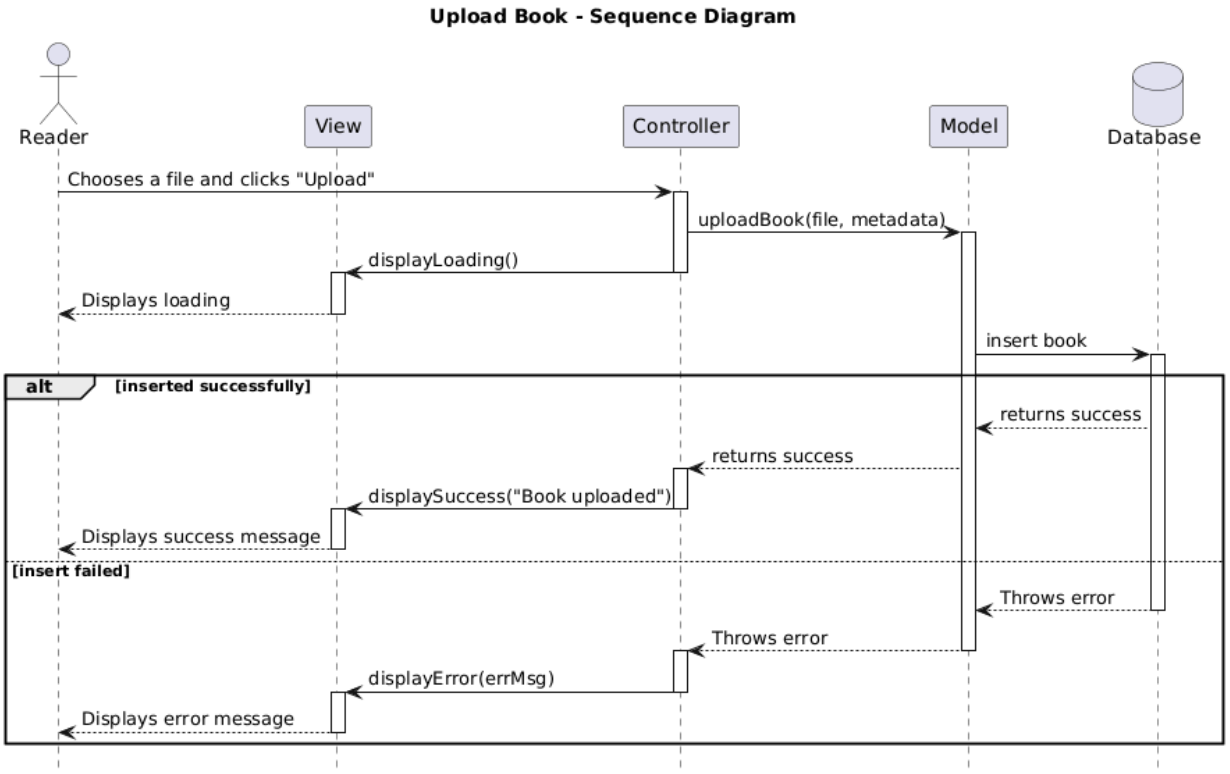
Use case diagram – Provide a use case diagram (Figure 5.5) for your project

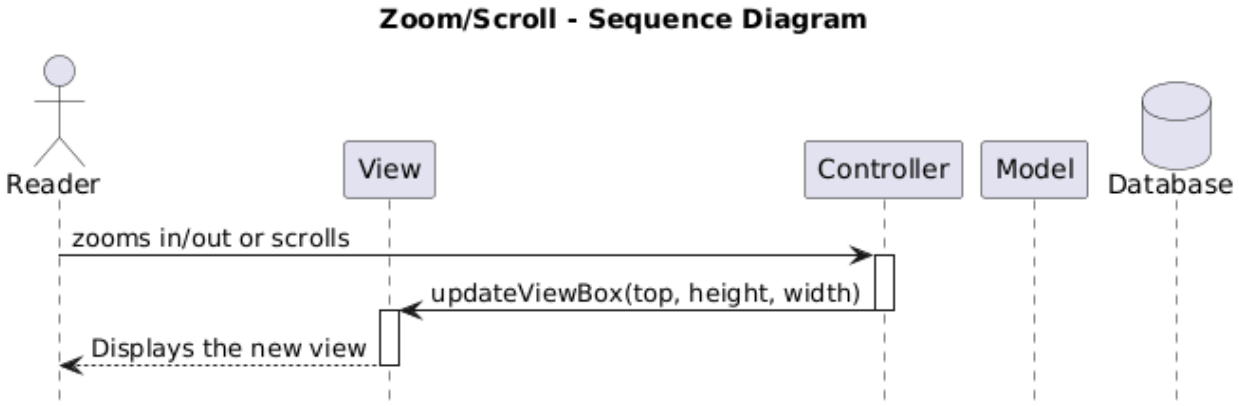
a. Please note that there should be more than one use case depending on the complexity of your project. (Ch 5 and Ch 7)



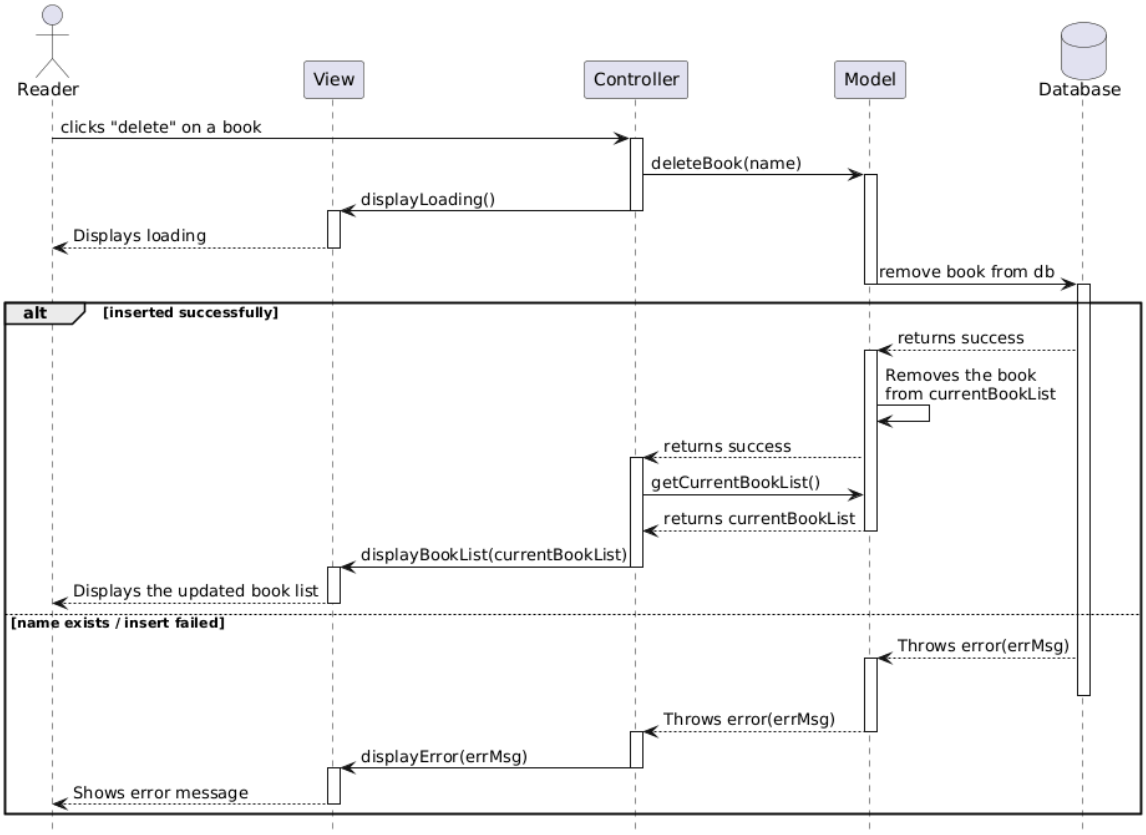
Sequence diagram – Provide sequence diagrams (Figure 5.6 and Figure 5.7) for each use case of your project.

a. Please note that there should be an individual sequence diagram for each use case of your project. (Ch 5 and Ch 7)

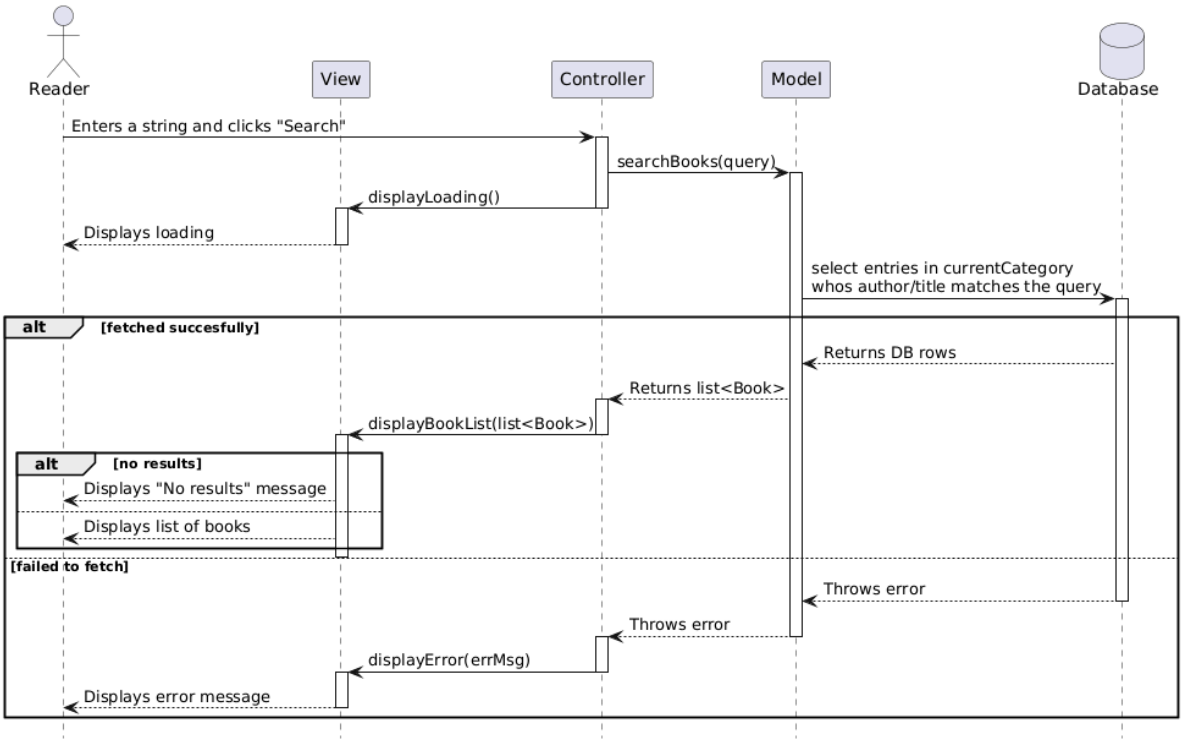




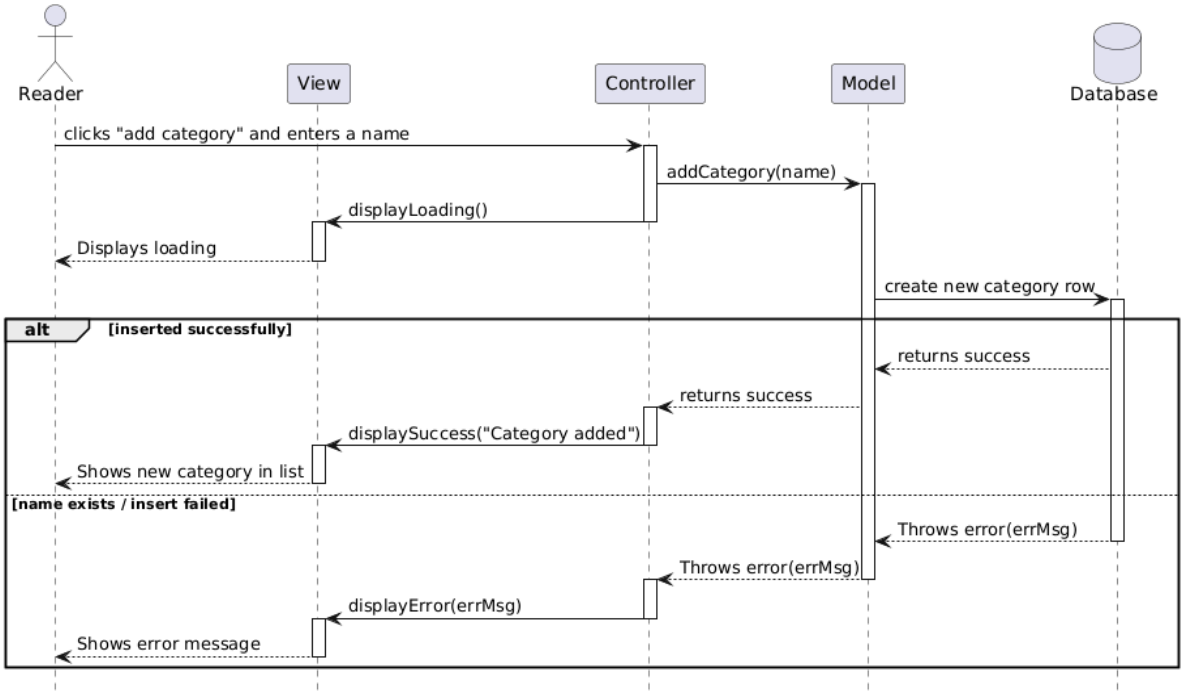
Delete Book - Sequence Diagram



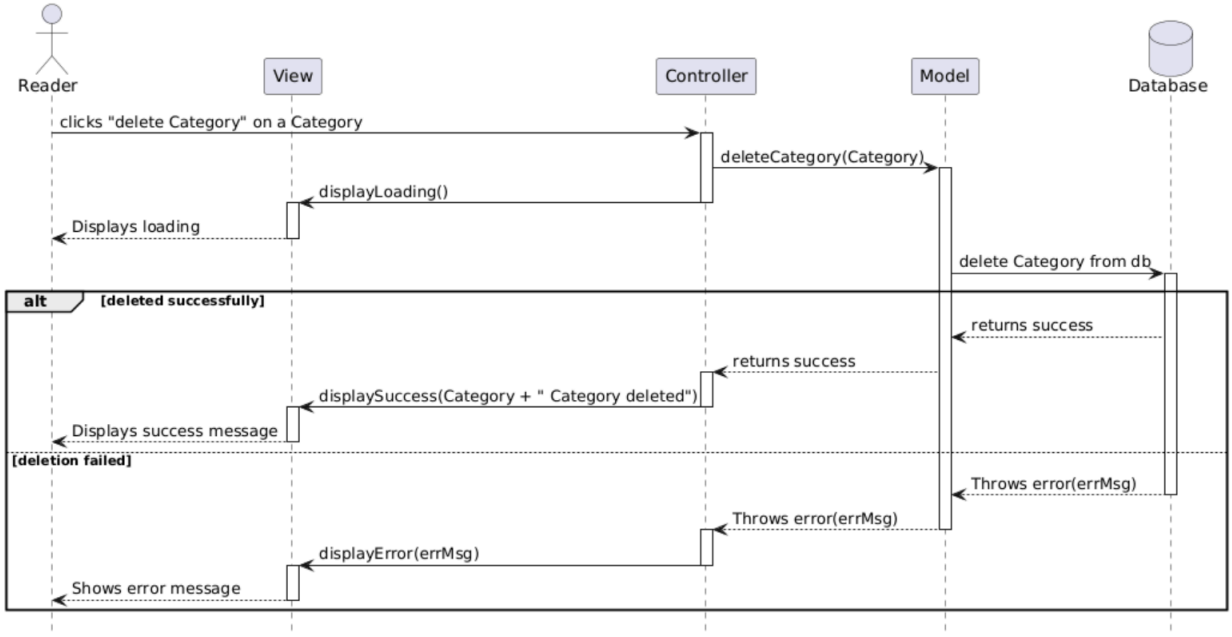
Search Books - Sequence Diagram



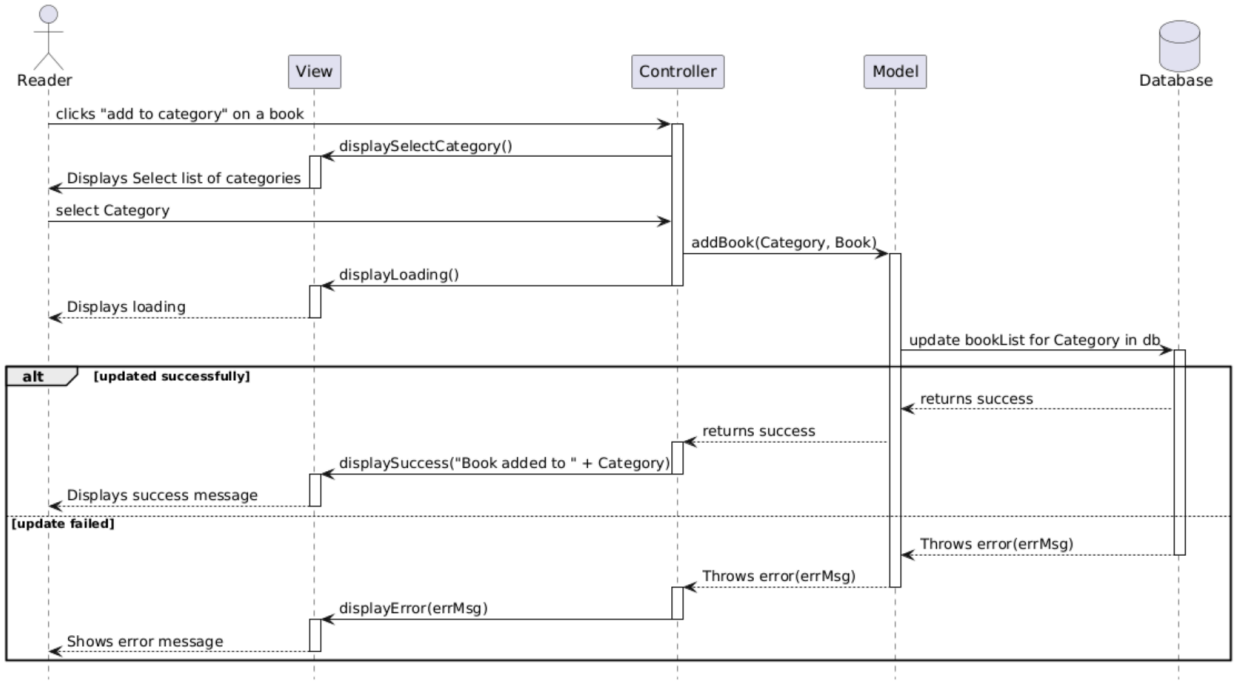
Add Category - Sequence Diagram



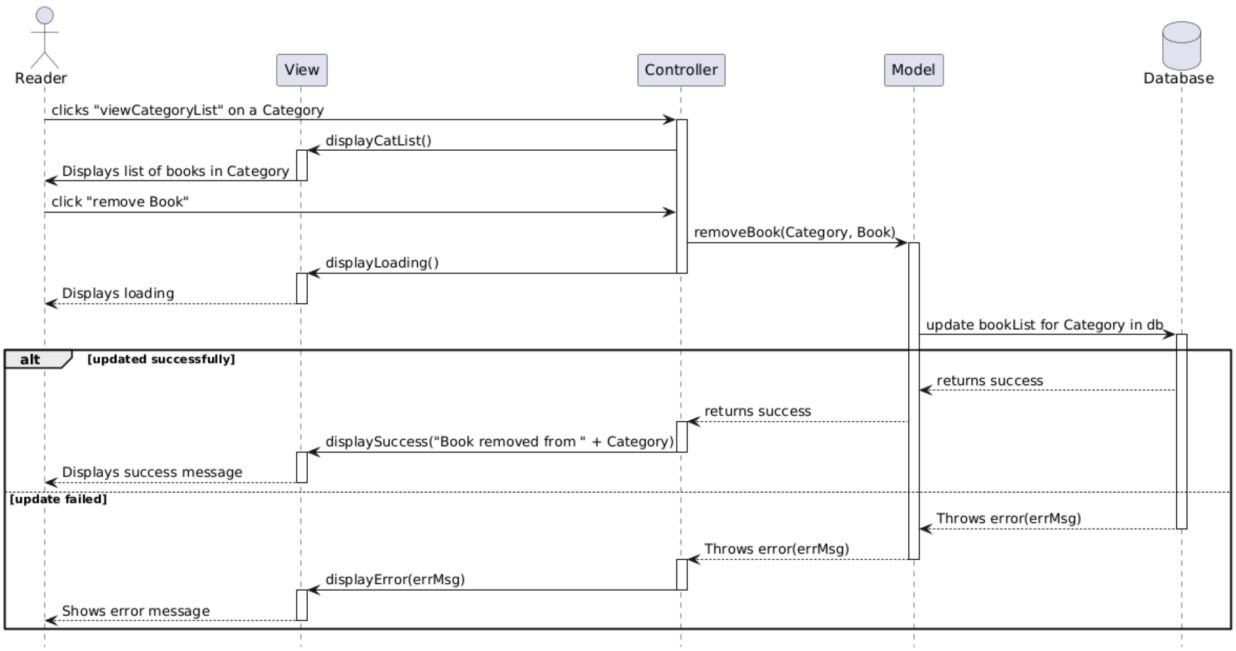
Delete Category - Sequence Diagram



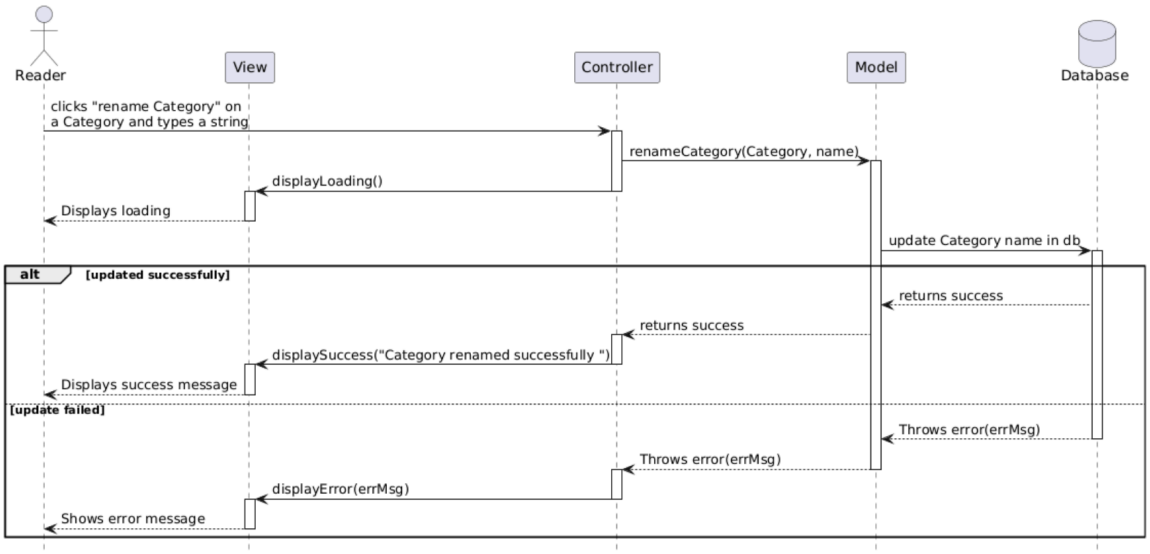
Add Book to Category - Sequence Diagram



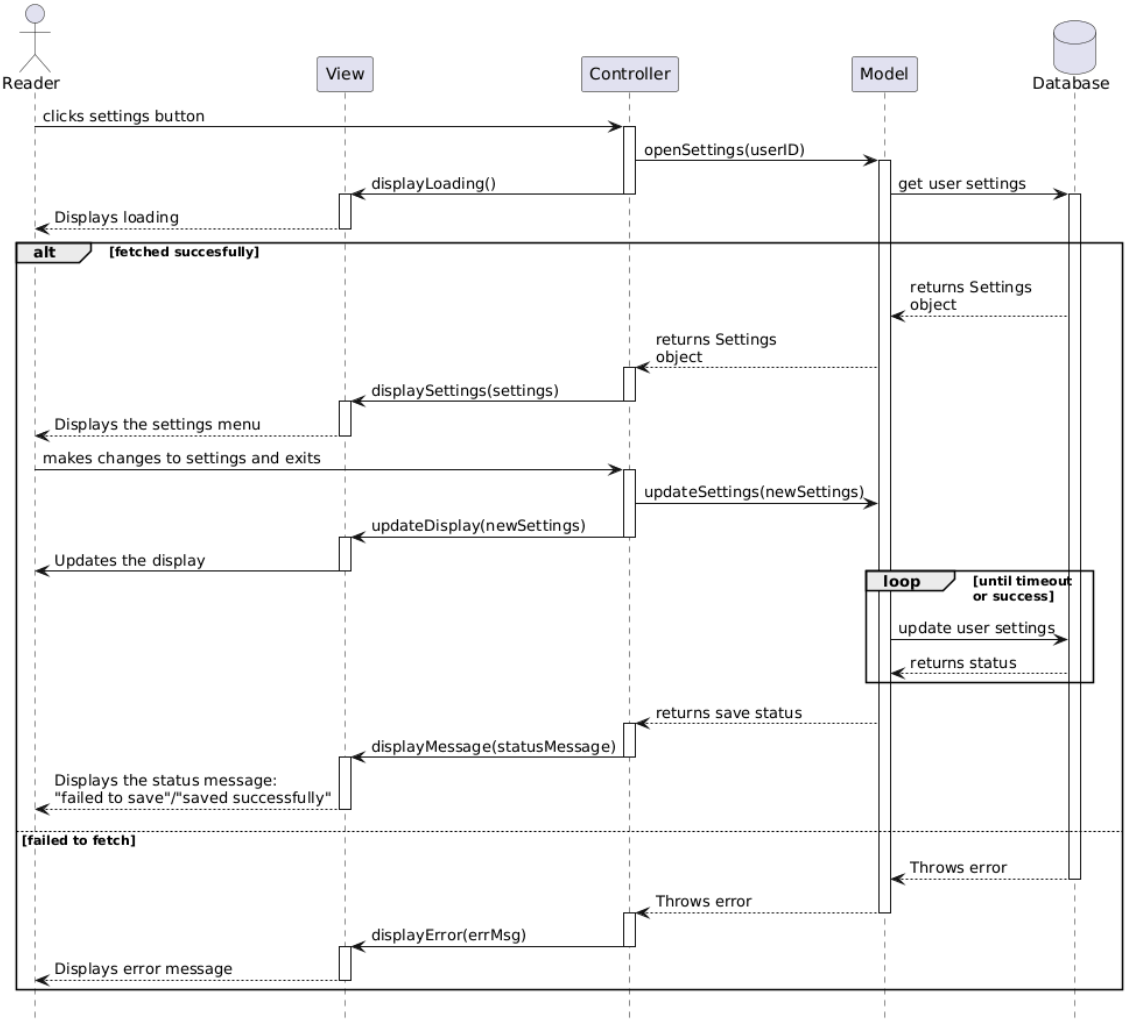
Remove Book from Category - Sequence Diagram



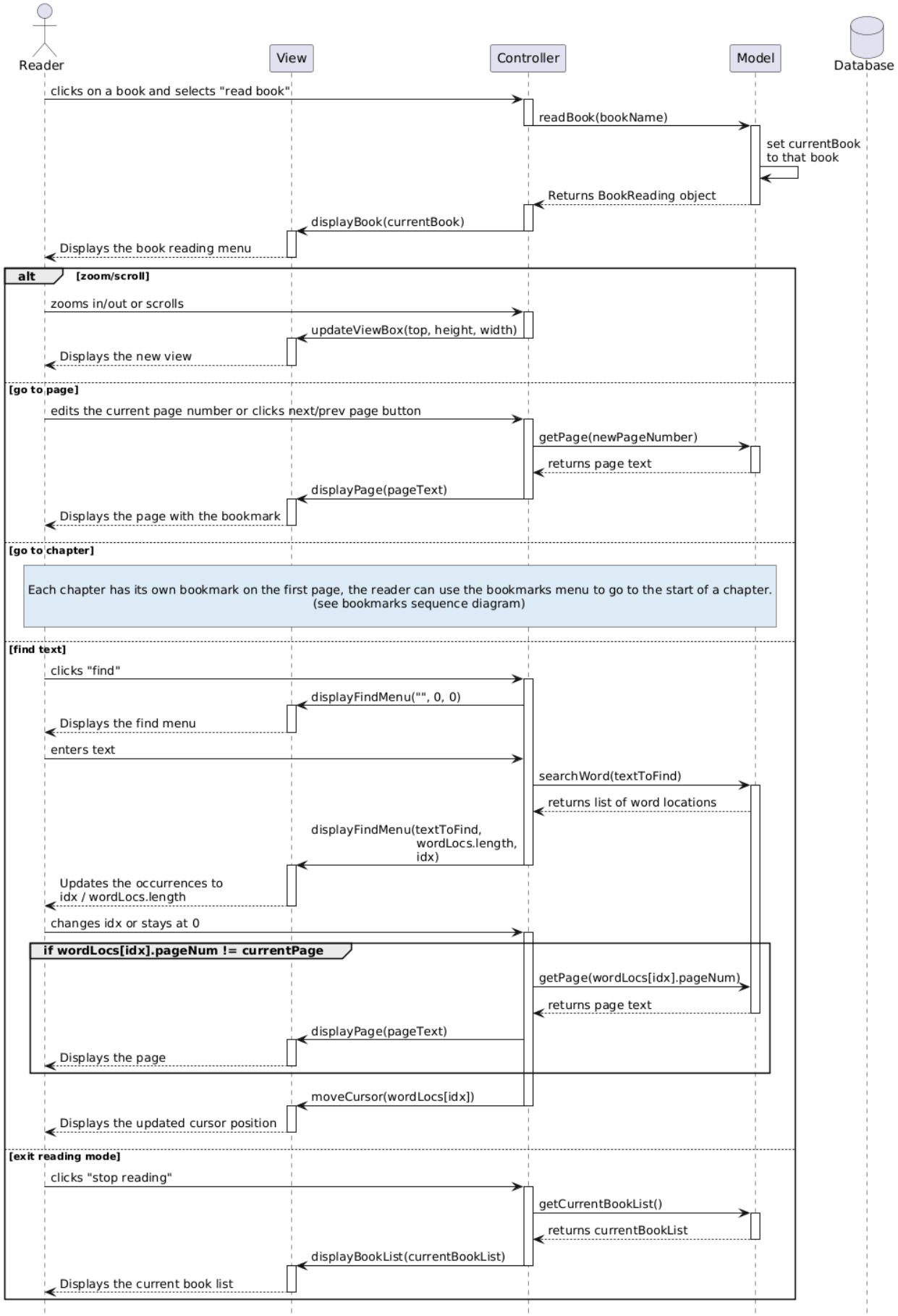
Rename Category - Sequence Diagram

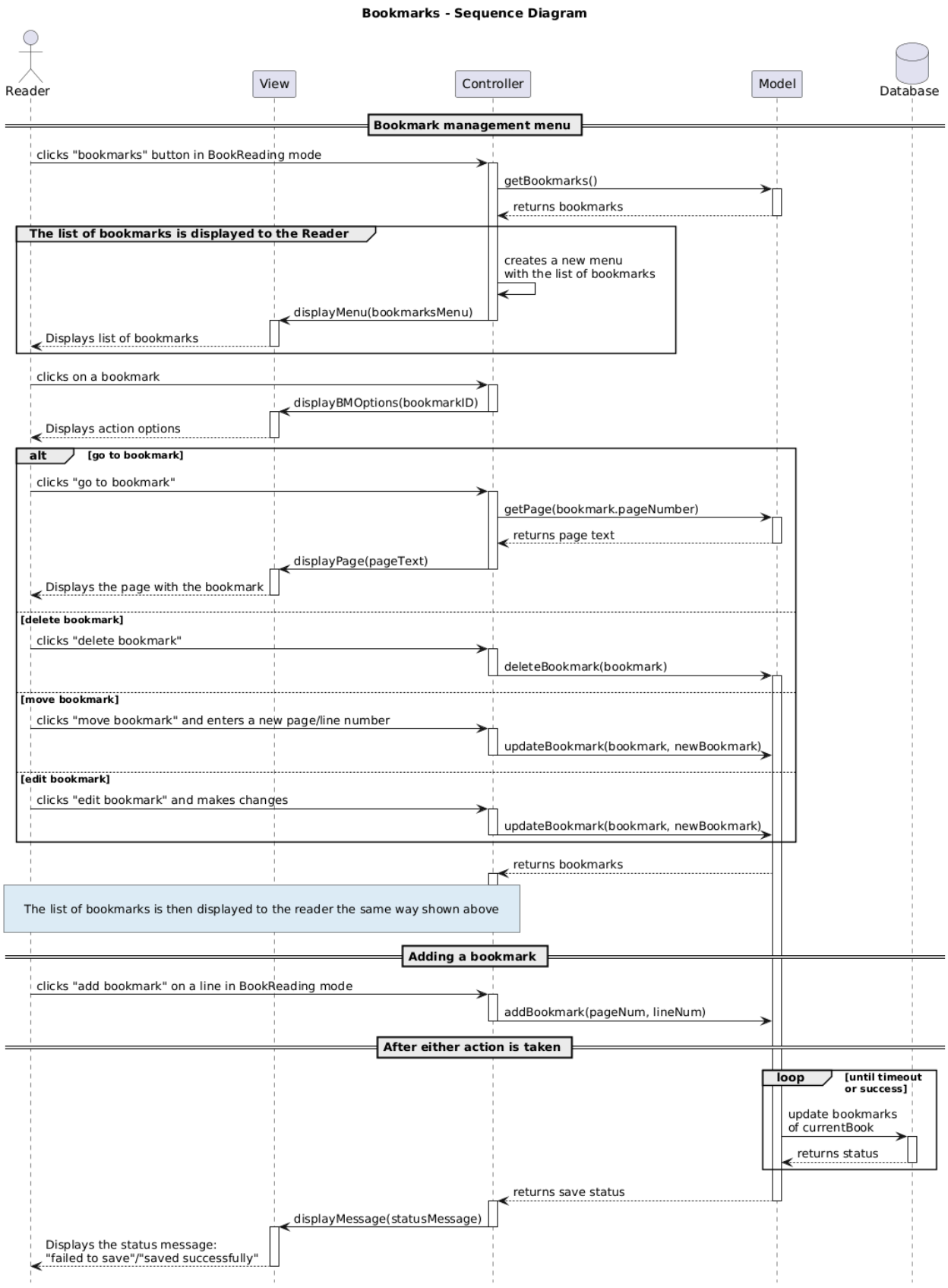


Adjust Settings - Sequence Diagram

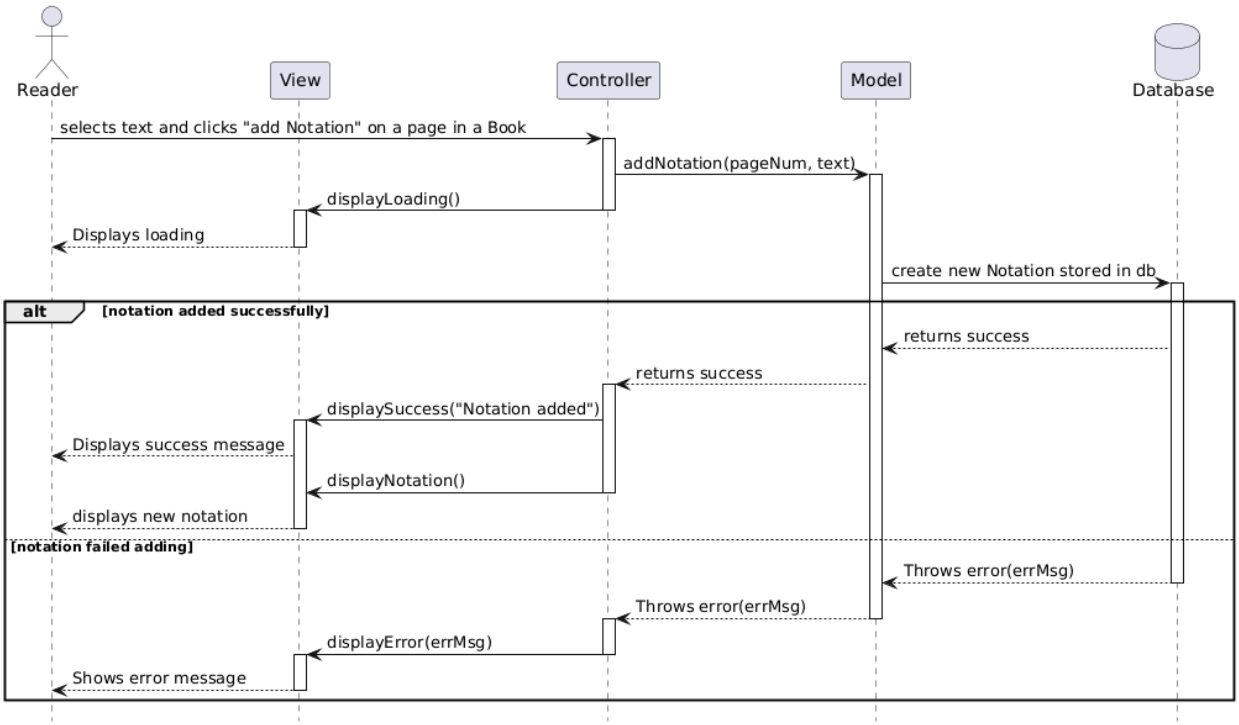


Read Book - Sequence Diagram

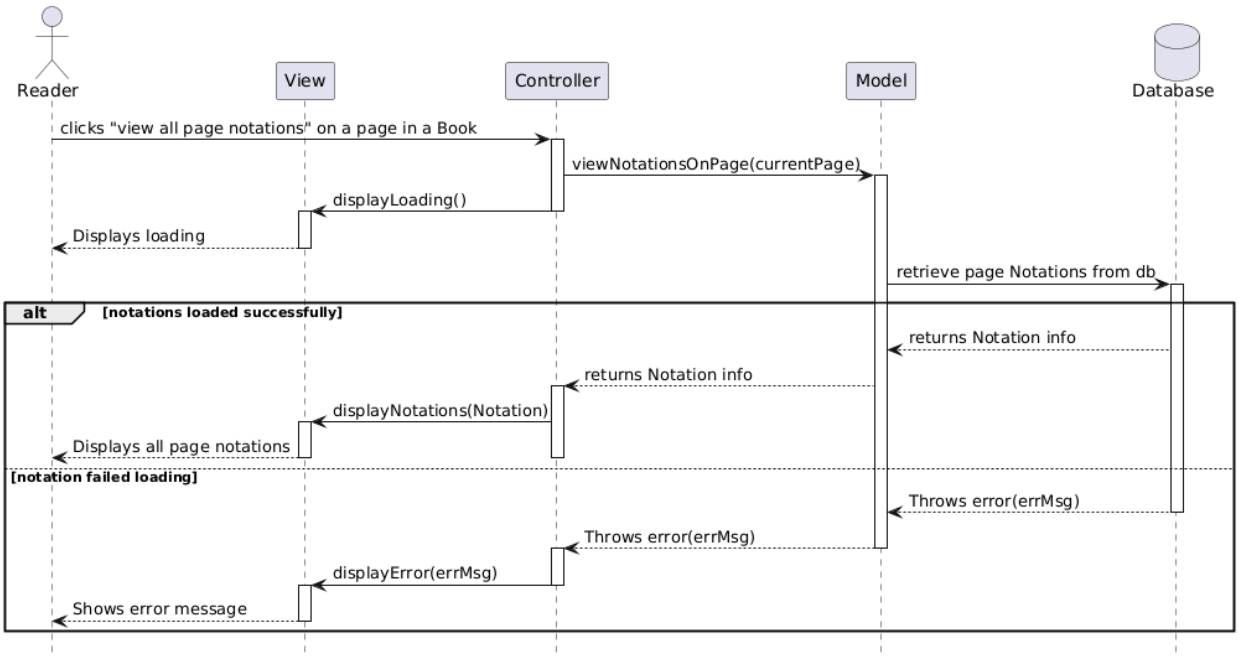



Bookmarks - Sequence Diagram
Reader
View
Controller
Model
Database
Bookmark management menu
clicks "bookmarks" button in BookReading mode
getBookmarks()
returns bookmarks
The list of bookmarks is displayed to the Reader
creates a new menu
with the list of bookmarks
displayMenu(bookmarksMenu)
Displays list of bookmarks
clicks on a bookmark
displayBMOptions(bookmarkID)
Displays action options
alt
[go to bookmark]
clicks "go to bookmark"
getPage(bookmark.pageNumber)
returns page text
displayPage(pageText)
Displays the page with the bookmark
[delete bookmark]
clicks "delete bookmark"
deleteBookmark(bookmark)
[move bookmark]
clicks "move bookmark" and enters a new page/line number
updateBookmark(bookmark, newBookmark)
[edit bookmark]
clicks "edit bookmark" and makes changes
updateBookmark(bookmark, newBookmark)
returns bookmarks
The list of bookmarks is then displayed to the reader the same way shown above
Adding a bookmark
clicks "add bookmark" on a line in BookReading mode
addBookmark(pageNum, lineNum)
After either action is taken
loop
[until timeout
or success]
update bookmarks
of currentBook
returns status
returns save status
displayMessage(statusMessage)
Displays the status message:
"failed to save"/"saved successfully"

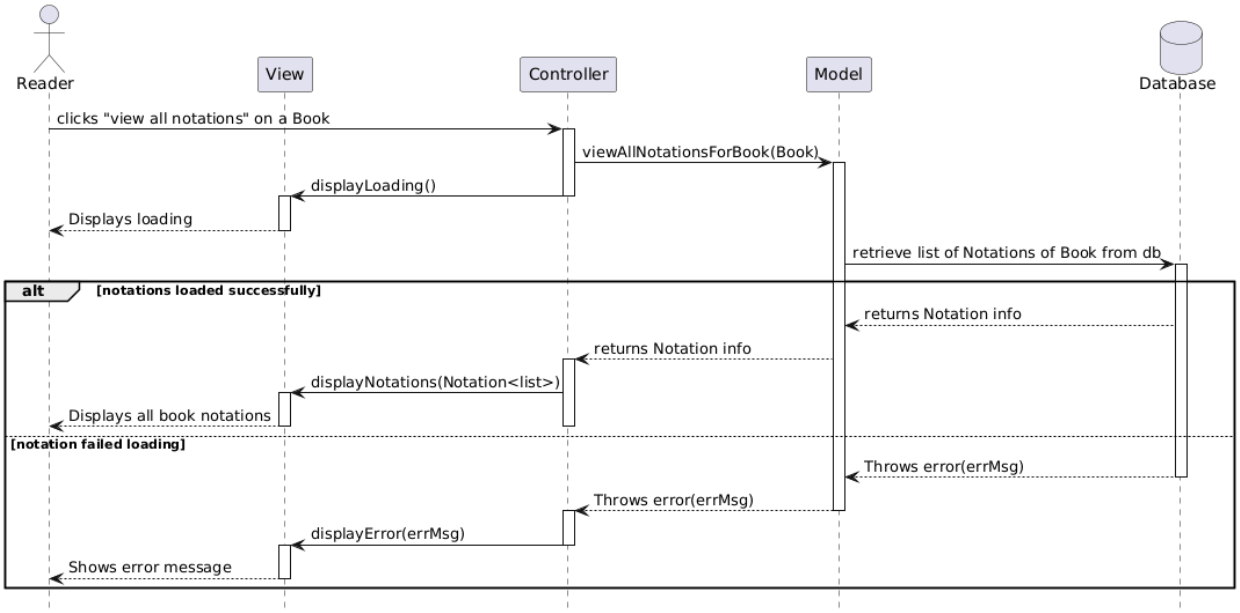
Add Notation - Sequence Diagram



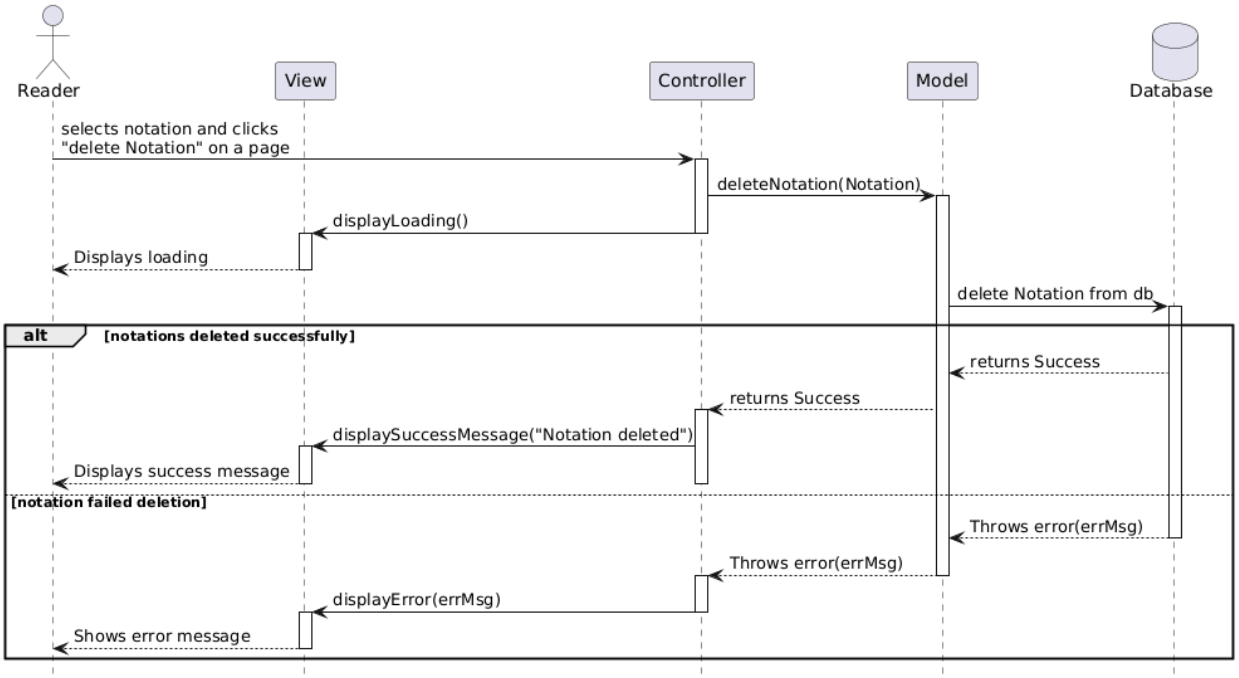
View Page Notations - Sequence Diagram



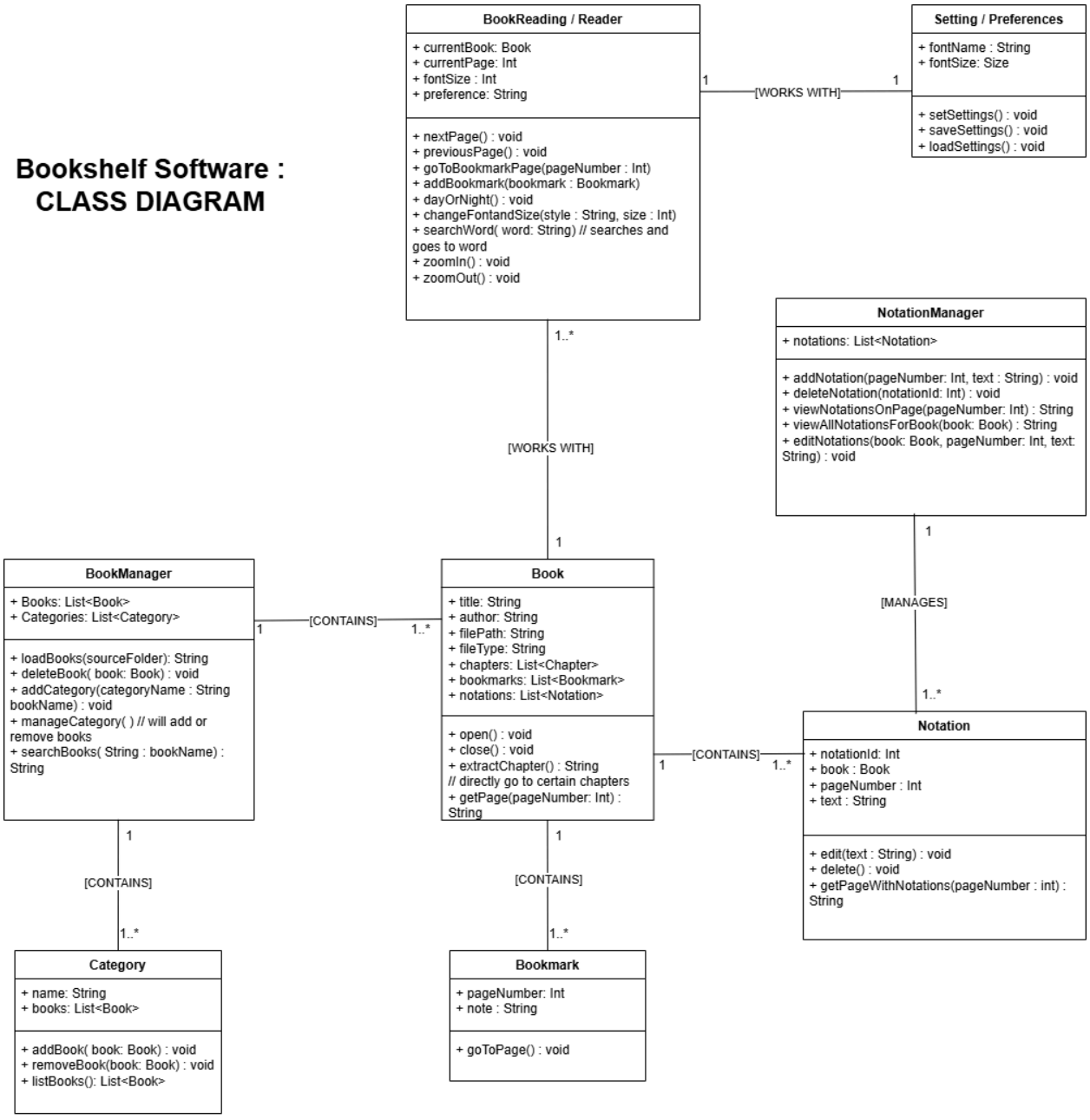
View all Book Notations - Sequence Diagram



Delete Notation - Sequence Diagram



Class diagram – Provide a class diagram of your project [3][4][7].



Architectural design – Provide an architectural design of your project [1][2].

a. Based on the characteristics of your project, choose, and apply only one appropriate architectural pattern from the following list: (Ch 6)

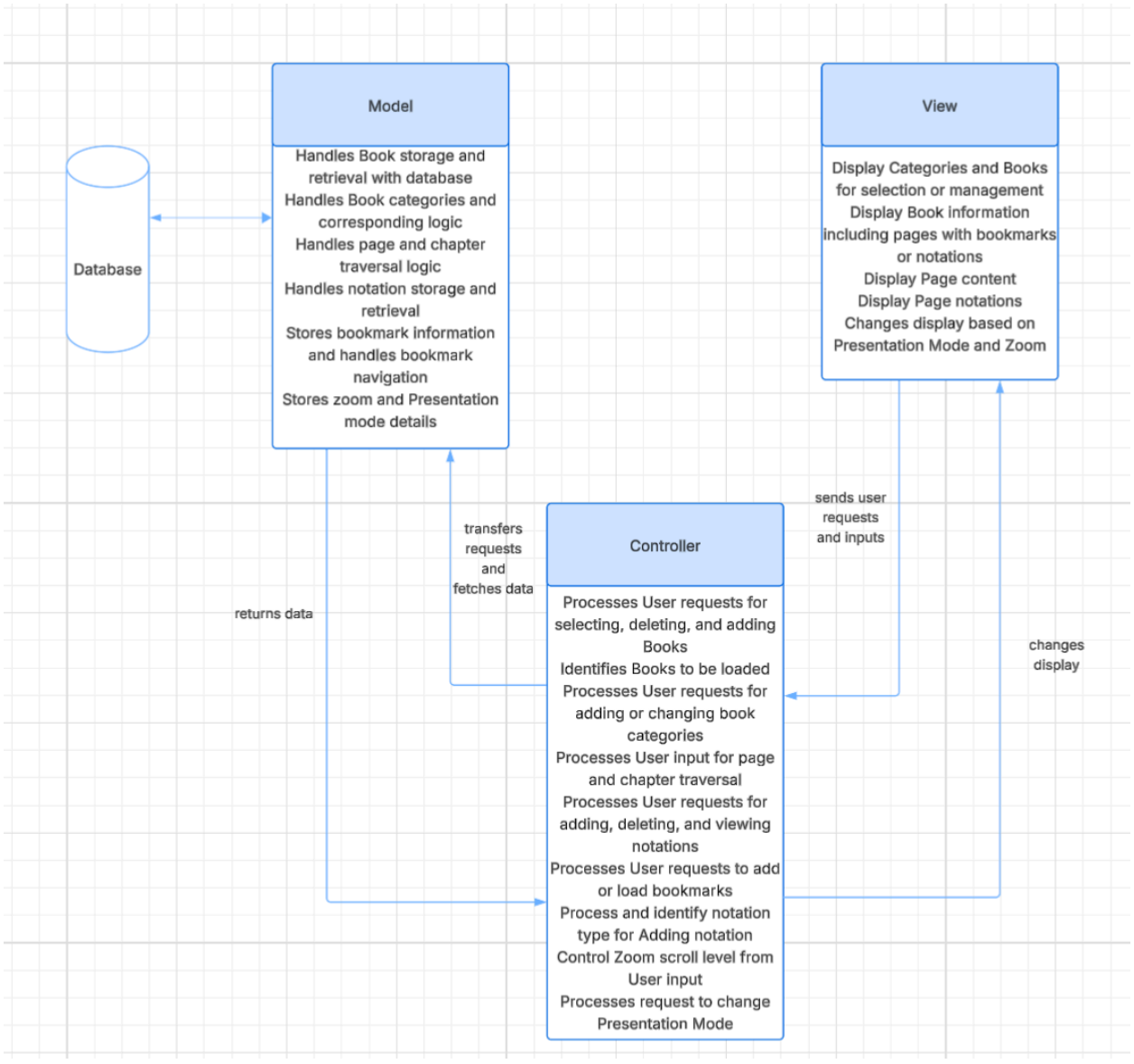
i. Model-View-Controller (MVC) pattern (Figure 6.6)

ii. Layered architecture pattern (Figure 6.9)

iii. Repository architecture pattern (Figure 6.11)

iv. Client-server architecture pattern (Figure 6.13)

v. Pipe and filter architecture pattern (Figure 6.15)



Project Scheduling:

i. Start and End Dates:

- After receiving feedback on the project proposal, and clarifying project requirements, we would start implementing the project on 01/05/2026.

- After taking 1 week to implement the project architecture, 1 week for UI prototyping, 2 weeks for implementation of the core features, 2 weeks for the advanced features, and 4 weeks for integration, testing, and documentation, the projected end date would be 03/16/2026

ii. Additional Details:

1. Weekends will not be included in the week structure

2. There will be 5 working hours per day per person

Cost, Effort and Pricing Estimation:

i. Estimated cost:

For this project we used a *simple effort-based cost estimation* method:

- We first fixed our **team size** at **7 developers** and assumed a **workload of 5 hours per person per week** (no weekends).
- We defined the **project scope** as only the features listed in our slides:
  - Management: load books (.txt, .pdf), delete books, create/manage categories, search books by text.
  - Reading: page navigation (swipe/next/previous), bookmarking, day/night mode, word search, basic chapter navigation.
  - Notation: add/view/edit/delete notes, view all notes in a book.
  - Quality of life: zoom in/out, optional scrolling mode.
- We broke the work into phases and assigned **approximate hours** to each phase:
  - 1 week architecture design – **25 hours**
  - 1 week UI prototyping – **25 hours**
  - 2 weeks core functionality – **50 hours**
  - 2 weeks advanced features – **50 hours**
  - 4 weeks integration / testing / documentation – **100 hours**
- **Total effort = 25 + 25 + 50 + 50 + 100 = 250 person-hours.**
- We used a blended “company” **hourly rate of $35/hour** for a junior / student development team.
- **Development cost = 250 hours × $35/hour = $8,750.**

- On top of pure development effort, we added:
    - A budget for third-party tools/libraries (PDF rendering, UI components, etc.)
    - A contingency and profit margin for the "company."

This gives a final **estimated total project cost in the range of about $10,000–$12,000**, which is the price we would quote to a client.

ii. Chosen cost modeling technique:

We align our estimate with the **Application Composition model**:

- The system is naturally divided into **high-level application objects** such as:
    - Library/Bookshelf view
    - Reader view (page navigation, word search, day/night mode)
    - Notes/Annotations view
    - Category management view
    - Settings / quality-of-life features
- Much of the functionality will be built by **composing existing components and libraries** (PDF viewer, UI frameworks, search utilities) rather than writing everything from scratch.
- For each major "object" (screen/module), we roughly allocated effort (UI + logic + testing) and summed them across phases. This object-based breakdown is what led to the **250 person-hour total**, which then fed directly into the effort-based cost calculation described above.

So, while the arithmetic is straightforward (hours × rate), the underlying **cost model is Application Composition**, because we estimated effort based on assembling and integrating predefined application components.

c. Estimated cost of hardware products (such as servers, etc.):

The application is light-weight and can run on standard infrastructure, so hardware costs are minimal:

- **Development / testing hardware:** assumed to use existing student machines – **$0 incremental cost.**
- **Production hosting (cloud VM / storage for ebooks and notes):**
    - Small cloud instance + storage for the first year: ≈ **$400–$500.**

For the estimate we round to: **Estimated hardware cost: ≈ $500.**

d. Estimated cost of software products (such as licensed software, etc.):

We assume mainly open-source tools, but budget for a few paid items:

- Possible **PDF/ebook rendering library license** or commercial SDK: ≈ **$300–$500.**
- Possible **UI/component library / design tools / project management tools** beyond free tiers: ≈ **$200–$300.**

So we allocate: **Estimated software/licensing cost: ≈ $500–$800**

e. Estimated cost of personnel (number of people to code the final product, training cost after installation):

Personnel cost covers **all development + basic training/hand-off after installation**:

- **Team:** 7 developers.
- **Total planned effort:** 250 person-hours (design, coding, testing, documentation, deployment, short training session for client).
- **Blended hourly rate:** $35/hour.
- **Personnel cost = 250 × $35 = $8,750.**

If we combine everything:

- Personnel: **$8,750**
- Hardware: ≈ **$500**
- Software/licenses: ≈ **$500–$1,000**
- Contingency + company overhead/profit: ≈ **10–20%**

We land at a **quoted project price of roughly $10,000–$12,000**,

Test Plan - Describe the test plan for testing a minimum of one unit of your software.

Following along with the Incremental software model, software testing follows a similar trajectory. Each additional feature and expansion will need to be tested for its proper use, improper use, and its edge cases. This process will be refined by use of automated testing. As an example of how a single unit may be tested, the following is a description of the testing done on the Bookshelf software's Category class.

As development proceeds, the automated tests will remain to speed up the process and ensure that the already established features of the Category class maintain proper functionality.

The Category class contains three methods that are yet to be integrated with the greater whole of the software. These methods are its addBook, removeBook, and listBook methods. Each will need to be tested by its intended use, unintended use, and actions performed at the edges.

addBook

1. Proper Use - Adding one to many books and verifying that it has those specific books in the category.
2. Improper Use – Adding duplicate books and verifying that it did not happen.
3. Edge Cases – Attempting to add something that is not a book and verifying that no issues arose.

removeBook

1. Proper Use – Remove one to many books that are inside the list and verifying that it is gone.
2. Improper Use – Removing a book that is not there and verifying nothing occurs.
3. Edge Cases – Attempting to remove something that is not a book and ensuring nothing happens.

listBook

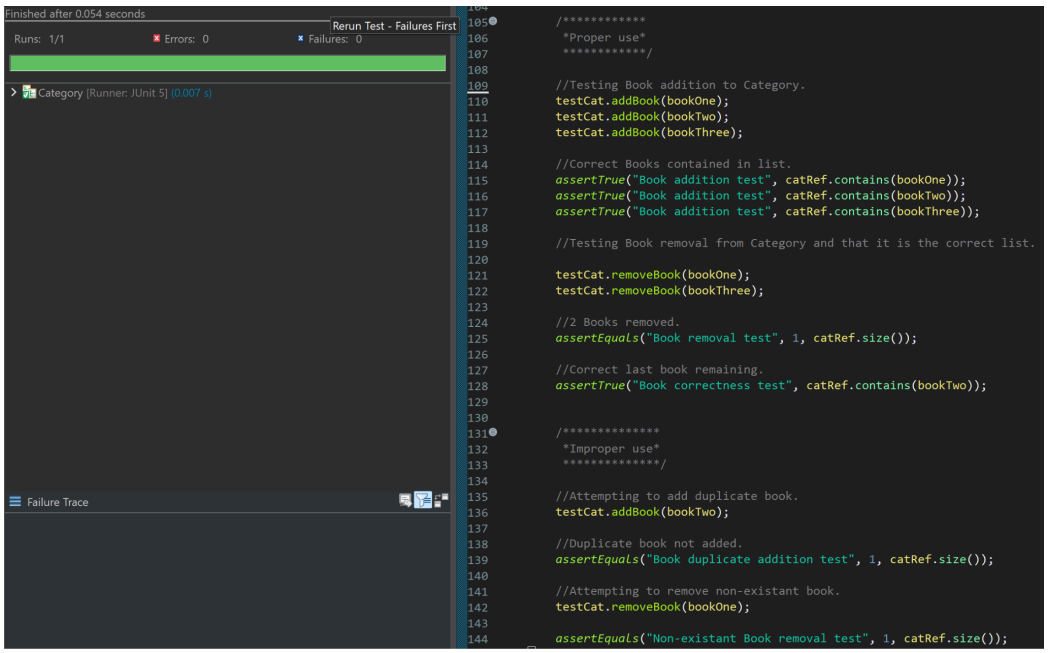
1. Proper Use – Returns the proper list reference of books and verifying the proper books are present.
2. Improper Use – None: This is meant to return a reference to the book list every time. There should be no improper use as it is separate from input and changing variables.
3. Edge Cases – None: edge cases involving obtaining a book list such as issues involving memory would not be tested here.

Proper Use Cases

Method	Test Cases	Result
addBook	Adding multiple books	Success - correct books present in list.
removeBook	Removing multiple books	Success - correct books removed from list.
listBooks	Obtaining proper list reference	Success - list verified as correct list.

Improper Use Cases

Method	Test Cases	Result
addBook	Adding duplicate books	Success - attempt was caught and was not performed.
removeBook	Removing non-existent books	Success - attempt was caught and was not performed.
listBooks	None	None - No changing factors prevent improper use.



Edge Cases

Method	Test Cases	Result
addBook	Attempting to add a non-book to the list	Compiler itself would not allow such an attempt.
removeBook	Attempting to remove a non-book from the list	Compiler itself would not allow such an attempt.
listBooks	None	None - related edge case tests would not be done here.

Comparison of your work with similar designs:

A thorough research of existing bookshelf softwares shows that our work shares core features with systems already out there but they share some differences as well. Some of the most notable bookshelf software in the market as of now include; Apple Books, Google Play Books, Kindle Books, Calibre, and VitalSource Bookshelf. Shifting our focus more on Google Play Books and Calibre, we will see how our software / work compares to them more in depth.

To start off with, what are the key components of our bookshelf software ? As iterated before, our work will focus on user experience with its simplicity and ease of use, having features such as: categories, bookmarks, notations, font management, theme management, and a search feature.

Google Play Books

Similarities:

Both our software and Google Play Books offer features for the user such as bookmarking, notations, customization of fonts, and display themes. These features aim to give the user a more personalized and customized experience whilst using the application.

Differences:

Based on their website [7] Google Play Books integrates the cloud into its software, for all the user's books to be stored and managed, everything is backed up to the cloud. It also uses online accounts such as gmail for their user recognition, according to their support page [8]. In addition, according to their website [6] it also provides a text-to-speech feature where they allow the user to listen to books read aloud. However, compared to our Software, Google Play Books does not support TXT files. It only accepts and supports PDF and EPUB files.

Conclusion:

Overall, compared to our software, Google Play Books seems to place more emphasis on the reading aspect of the bookshelf software while our software makes sure that our users get a more simple and easy experience when using our application. Our software places more emphasis on offering a consistent reading experience and interaction

Calibre E-Book Management Software

Similarities:

Based on the official website of Calibre [5] both our software and Calibre offer features such as book management, categories, searching for title or keywords, bookmarks, and fonts. Both are designed to help users maintain their documents effectively.

Differences:

Calibre mentions on their website [5] that it includes a metadata management system, which allows users to download all the metadata of a book (things like title, author, publisher, ISBN, tags, cover, summary) and allows you to edit those existing fields as well. It also provides features such as text-to-speech and a built-in plugin explorer [5]. It is also open source, for people to use.

Conclusion:

Compared to our software, Calibre is offering complex management and customization with their software, while we are prioritizing the user experience. Our software focuses more on simplicity and usability for our users.

Overview of Core Features Compared

	Our Software	Google Play Books	Calibre
Purpose / How the software works	Simple, easy use, file based Library management system	Cloud Based Library Management system, google ecosystem	Open-Source, metadata based Library Management system
UI Complexity	Simple & user friendly	Simple & dependent on google ecosystem	Advanced, offers many features
Supports TXT	Yes	Not really	Yes
Supports PDF	Yes	Yes	Yes
Categories	Yes	Yes	Yes
BookMarks	Yes	Yes	Yes
Search Books/Words	Yes	Yes	Yes
Day/Night Mode	Yes	Yes	Yes, as of recently
Font Manager	Yes	Yes	Yes
Notation Manager	Yes	Yes	Yes

Final Findings

While our software is not something extremely innovative, as seen from the very similar features that our software shares with already established ones, ours offers a balanced, simple, and easy usability for our target audience. Our target audience being students or faculty members that want an easy application where they can store all of their documents while simultaneously allowing them to use many management features as well. Overall, the primary distinction between ours and competing softwares is our focus on simplicity and ease of file handling rather than the complexity of metadata or cloud based control systems / management that other software like Calibre and Google Play Books offer.

Conclusion:

The overall planning of implementation that we had set in deliverable one, carried its way over to our final plan. We did not encounter any need for big changes that moved us away from our MVC pattern or our incremental process model.

We feel that the lack of changes is a result of the flexibility of what we had in our original plan and mainly due to two main things:

1. **MVC Pattern:** The pattern we chose for our software development has a strong separation of concerns which makes it simple to define test plans for individual components, without bringing up major flaws in the design logic.

2. **Incremental Process Model:** This model that we chose is designed to handle and prioritize work. We felt it fit perfectly with how things were being developed according to our project schedule. It allowed us to align core features and advanced features within their respective stages and increments.

In addition to this, our cost estimation confirmed that the project scope fits very well within the timeframe we have planned.

Works Cited:

[1] GeeksforGeeks, "MVC Design Pattern," *GeeksforGeeks*, Aug. 18, 2017. http://www.geeksforgeeks.org/system-design/mvc-design-pattern/ (Accessed Oct. 2025).

[2] GeeksforGeeks, "MVC Architecture System Design," *GeeksforGeeks*, Jul. 2024. http://www.geeksforgeeks.org/system-design/mvc-architecture-system-design/ (Accessed Oct. 2025).

[3] GeeksforGeeks, "UML Class Diagram," *GeeksforGeeks*, Aug. 30, 2018. http://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/ (Accessed Oct. 2025).

[4] ChatGPT, "*Examples of class diagrams for softwares*" *prompt,* OpenAI, [Online]. chat.openai.com/chat (Accessed Oct. 2025)

[5] "calibre - About," *calibre-ebook.com*.[Online]. Available:https://calibre-ebook.com/about (Accessed Nov.2025)

[6] "Google Play Books App Overview," *The DAISY Consortium*. [Online]. Available: https://daisy.org/guidance/info-help/guidance-training/reading-systems/google-play-books-app-overview/ (Accessed Nov. 2025)

[7] "Google Play," *Google.com*, 2025.[Online]. Available: https://play.google.com/intl/en-GB_uk/about/books/user-policies-for-google-play-books/ (Accessed Nov. 2025).

[8] "Security and content protection for books on Google Play - Google Play Books Partner Center Help," *Google.com*, 2019. [Online]. Available: https://support.google.com/books/partner/answer/170679?hl=en (Accessed Nov. 2025).

[9] I. Sommerville, *Software engineering*, 10th ed. Boston: Pearson Education Limited, 2016.